Trabalho Prático de Gestão de Configuração

Simulação de desenvolvimento colaborativo com Git e GitHub

Curso: Sistemas de Informação
Disciplina: Engenharia de Software
Professora: Adriana
Instituição: UNASP
Grupo: Projeto-GCS
Data: Novembro de 2026

Integrantes e responsabilidades

Integrante	Papel	Responsabilidades
Thiago Duarte	Gestor de Configuração	Administrar branches, tags, releases, proteção da branch principal e organização do relatório.
Crisnelly Nobre	Desenvolvedora principal	Implementar as funcionalidades principais e manter a branch `develop`.
Leandro Pires	Revisor de código	Revisar pull requests, verificar padrões e aprovar alterações.

> Como o grupo possui três integrantes, todos também podem atuar em atividades de desenvolvimento, mantendo a separação entre implementação, revisão e gestão da configuração.

---

1. Introdução

A Gestão de Configuração de Software, ou GCS, reúne práticas para identificar, controlar, acompanhar e auditar os artefatos produzidos durante o desenvolvimento de um sistema. Essas práticas são importantes porque permitem saber qual versão de cada arquivo está sendo utilizada, quem realizou determinada alteração, por que a alteração foi feita e como recuperar uma versão anterior quando necessário.

Neste trabalho foi simulada a utilização do Git e do GitHub em um projeto colaborativo denominado **TaskFlow**, um sistema simples de lista de tarefas. O projeto foi escolhido por possuir escopo pequeno, mas permitir a demonstração dos principais conceitos de GCS: repositório compartilhado, branches independentes, commits padronizados, pull requests, revisão de código, conflito intencional, resolução de conflito, merge, tags e recuperação de versões.

O objetivo principal foi reproduzir um ambiente de desenvolvimento com múltiplos participantes, aplicando uma política de configuração definida antes da implementação.

---

2. Descrição do projeto

O TaskFlow é uma aplicação simples para gerenciamento de tarefas pessoais ou de uma equipe. A primeira versão contempla as seguintes operações:

- cadastrar uma tarefa;
- listar tarefas cadastradas;
- marcar uma tarefa como concluída;
- remover uma tarefa;
- documentar a forma de instalação e utilização.

O escopo reduzido facilita a simulação das atividades de configuração sem retirar a necessidade de colaboração entre os integrantes.

2.1 Estrutura inicial do repositório

```
TaskFlow/
├── README.md
├── LICENSE
├── .gitignore
├── docs/
│   └── processo-gcs.md
├── src/
│   ├── main.py
│   └── tarefas.py
├── tests/
│   └── test_tarefas.py
└── .github/
    └── pull_request_template.md
```

2.2 Itens de configuração

Foram definidos como itens de configuração todos os artefatos que podem influenciar o funcionamento, a instalação, a documentação ou o controle do projeto:

Item	Identificação	Controle aplicado
Código-fonte	`src/`	Versionamento pelo Git e revisão via pull request.
Testes	`tests/`	Alterações associadas às funcionalidades correspondentes.
Documentação	`README.md` e `docs/`	Revisão textual e atualização junto às mudanças relevantes.
Dependências	`requirements.txt`, se necessário	Alterações registradas em commit separado.
Configuração do Git	`.gitignore` e `.github/`	Mantida sob revisão do gestor de configuração.
Releases	Tags e notas de versão	Criadas somente a partir da branch `main`.

3. Planejamento da configuração

3.1 Convenções de nomeação

Foram adotadas as seguintes convenções:

- nomes de branches em letras minúsculas;
- palavras separadas por hífen;
- nomes de arquivos em letras minúsculas;
- classes e funções seguindo o padrão da linguagem utilizada;
- tags seguindo o versionamento semântico;
- pull requests com título objetivo e relacionado à funcionalidade implementada.

Exemplos:

```
feature/cadastro-tarefa
feature/concluir-tarefa
fix/validacao-tarefa
docs/atualiza-readme
release/v1.0.0
```

3.2 Convenção de commits

Foi adotado o padrão **Conventional Commits**, com a seguinte estrutura:

```
<tipo>(<escopo>): <descrição no imperativo>
```

Tipos utilizados:

Tipo	Utilização
`feat`	Inclusão de uma nova funcionalidade.
`fix`	Correção de um defeito.
`docs`	Alteração na documentação.
`test`	Inclusão ou alteração de testes.
`refactor`	Reestruturação sem mudança de comportamento.
`chore`	Tarefa de manutenção ou configuração.
`build`	Mudança em dependências ou processo de build.

Exemplos de commits válidos:

```
feat(tarefas): adiciona cadastro de tarefas
feat(tarefas): permite concluir tarefa
fix(validacao): impede cadastro com titulo vazio
test(tarefas): adiciona testes de conclusao
docs(readme): explica como executar o projeto
chore(git): adiciona configuracao de ignorados
```

As mensagens devem ser curtas, objetivas e escritas no presente do indicativo ou no imperativo. Commits genéricos como `alterações`, `coisas`, `versão final` ou `ajustes` não devem ser utilizados.

3.3 Política de versionamento

Foi adotado o **Versionamento Semântico**, no formato:

```
MAJOR.MINOR.PATCH
```

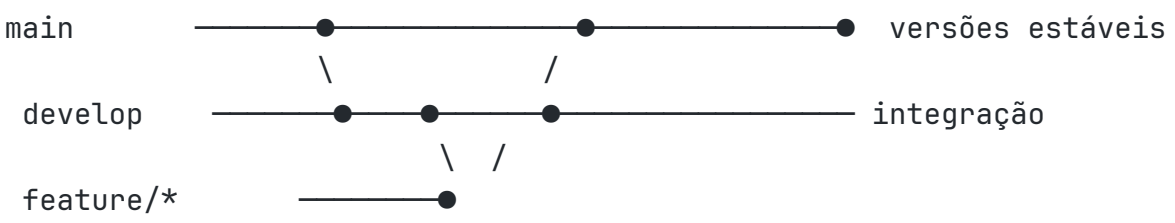
- **MAJOR:** alteração incompatível com a versão anterior;
- **MINOR:** inclusão de funcionalidade compatível;
- **PATCH:** correção compatível de defeitos.

Exemplos:

- `v1.0.0`: primeira versão estável do TaskFlow;
- `v1.1.0`: inclusão de filtro ou prioridade de tarefas;
- `v1.1.1`: correção de um erro de validação;
- `v2.0.0`: mudança incompatível na estrutura ou na interface.

3.4 Política de branching

O fluxo adotado foi baseado em uma versão simplificada do Git Flow:



Regras:

1. `main` contém somente versões estáveis e publicáveis.
2. `develop` reúne as funcionalidades aprovadas para a próxima versão.
3. Cada funcionalidade deve ser desenvolvida em uma branch `feature/...`.
4. Correções urgentes devem ser feitas em branches `fix/...`.
5. Nenhum integrante deve enviar alterações diretamente para `main`.
6. Toda alteração deve chegar a `develop` por meio de pull request.
7. O pull request deve possuir descrição, revisão e verificação dos testes.
8. Após a validação da versão, `develop` é mesclada em `main` e recebe uma tag.

3.5 Estratégia de backup e recuperação

O repositório remoto no GitHub é o backup principal do projeto. Além dele, foram definidas as seguintes práticas:

- manter o repositório remoto privado ou público conforme a orientação do professor;
- utilizar mais de um colaborador com permissão administrativa;
- realizar clone local pelos integrantes;

- criar tags para as versões entregues;
- utilizar releases do GitHub para registrar versões estáveis;
- exportar periodicamente o repositório por meio de `git bundle`;
- evitar armazenar senhas, tokens e arquivos gerados no repositório;
- recuperar uma versão por meio de uma tag ou do histórico de commits.

Exemplo de backup completo:

```
git bundle create taskflow-backup.bundle --all
```

Exemplo de recuperação:

```
git clone taskflow-backup.bundle TaskFlow-recuperado
```

---

4. Criação do repositório e configuração inicial

O repositório foi criado no GitHub com o nome **TaskFlow**. Após a criação, foram adicionados o README, o `.gitignore`, a licença e a estrutura inicial de diretórios.

> **Preencher antes da entrega:** substituir os campos abaixo pelas informações reais do grupo.
>
> - URL do repositório: `[colar aqui o link do GitHub]`
> - Organização ou proprietário: `[usuário ou organização]`
> - Visibilidade: `[público/privado]`
> - Data de criação: `[data]`

Comandos utilizados na configuração local:

```
mkdir TaskFlow
cd TaskFlow
git init
git branch -M main
git remote add origin https://github.com/USUARIO/TaskFlow.git
mkdir -p src tests docs .github
touch README.md LICENSE .gitignore

git add .
git commit -m "chore(repo): cria estrutura inicial do projeto"
git push -u origin main

git checkout -b develop
git push -u origin develop
```

4.1 Proteção das branches

No GitHub, a branch `main` foi configurada para exigir:

- pull request antes do merge;
- pelo menos uma aprovação de revisor;
- execução dos testes antes da integração;
- proibição de push direto, quando essa opção estiver disponível no plano utilizado.

A branch `develop` foi utilizada como área de integração das funcionalidades aprovadas.

---

5. Controle de versão e colaboração

Cada integrante trabalhou em uma branch própria, criada a partir de `develop`. O fluxo de trabalho utilizado foi:

```
git checkout develop
git pull origin develop
git checkout -b feature/minha-funcionalidade

# editar os arquivos
git status
git add .
git commit -m "feat(escopo): descreve a alteração"
git push -u origin feature/minha-funcionalidade
```

Depois do `push`, o integrante criou um pull request no GitHub apontando para `develop`. O revisor verificou:

- clareza do código;
- aderência à convenção de nomes;
- existência de testes;
- atualização da documentação;
- resultado da execução dos testes;
- ausência de arquivos indevidos ou informações sensíveis.

Após a aprovação, o pull request foi integrado por merge e a branch de trabalho foi excluída.

---

6. Simulação e resolução de conflito

Durante o desenvolvimento do projeto, ocorreu um conflito relacionado ao título da página no arquivo `src/index.html`. Duas alterações modificaram a mesma região do arquivo e, após a integração, o título apareceu duplicado. Esse problema foi utilizado para demonstrar a identificação, a correção e o registro de um conflito de merge.

6.1 Alterações conflitantes

As branches `feature/titulo-gestor` e `feature/titulo-principal` alteraram a região do título da página. Como as duas alterações atuaram sobre o mesmo trecho de `src/index.html`, a integração produziu uma duplicação do título. O conflito foi identificado durante a integração das contribuições e exigiu uma correção específica antes da conclusão do merge.

6.2 Branch e commit da correção

Para corrigir o problema, foi criada a branch `fix-remove-título-duplicado-após-conflito`. Nessa branch, o título duplicado foi removido e apenas a versão correta foi mantida.

O commit que registrou a correção foi:

```
2be62f4 — fix/titulo-duplicado
```

Depois, a correção foi integrada por meio do pull request número 4:

```
9223ea8 — Merge pull request #4 from ThiagoDuarteDS/fix-remove-título-dup
```

Assim, o conflito foi resolvido, a alteração foi revisada e a versão corrigida foi integrada à branch `develop`.

6.3 Resultado

A resolução foi validada verificando o arquivo `src/index.html` após o merge. O título duplicado deixou de aparecer. A atividade demonstrou que o Git identifica alterações incompatíveis, mas a decisão sobre qual versão deve permanecer depende da análise dos integrantes.

---

7. Tags e releases

Após a integração das funcionalidades da primeira versão, foi criada a tag `v1.0.0`:

```
git checkout main
git pull origin main
git merge --no-ff develop -m "release: prepara versao 1.0.0"
git tag -a v1.0.0 -m "Primeira versao estavel do TaskFlow"
git push origin main --tags
```

No GitHub, a tag foi utilizada para criar uma release com as seguintes informações:

TaskFlow v1.0.0

Primeira versão estável do sistema de lista de tarefas. Inclui cadastro, listagem, conclusão e remoção de tarefas, além da documentação inicial e dos testes básicos.

---

8. Histórico de commits

O histórico abaixo representa o padrão esperado no repositório:

```
f3a8c21 (tag: v1.0.0, main) release: prepara versao 1.0.0
b82d190 Merge pull request #4 from grupo/develop
4c7a112 fix(readme): resolve conflito na descricao do projeto
9d1e4ab Merge pull request #3 from grupo/feature/concluir-tarefa
6a4e8f0 feat(tarefas): permite concluir tarefa
2b9c771 test(tarefas): adiciona testes de conclusao
1e5d032 Merge pull request #2 from grupo/feature/cadastro-tarefa
7f2a901 feat(tarefas): adiciona cadastro de tarefas
0d4be88 chore(repo): cria estrutura inicial do projeto
```

O histórico real deve ser obtido no repositório do grupo com:

```
git log --oneline --decorate --graph --all
```

> **Inserir aqui o print real do histórico de commits do GitHub ou do terminal.**
>
> **Figura 1 – Histórico de branches, merges e tags.**
> `[INSERIR PRINT AQUI]`

---

9. Dificuldades encontradas e soluções

Dificuldade	Solução adotada
Alterações feitas diretamente na branch principal	Definição de fluxo com `feature/*`, `develop` e pull requests.
Mensagens de commit pouco claras	Adoção do Conventional Commits e revisão do histórico.
Duas alterações modificaram a mesma região de `src/index.html`, causando duplicação do título	Criação da branch `fix-remove-título-duplicado-após-conflito`, remoção do título duplicado e registro da correção no commit `2be62f4`.
Dúvida sobre a versão entregue	Criação da tag `v1.0.0` e da release correspondente.
Risco de perda do trabalho local	Uso do GitHub como repositório remoto e criação de backup com `git bundle`.
Falta de documentação de uma funcionalidade	Inclusão da atualização do README como critério de revisão do pull request.

---

10. Evidências da execução

As evidências devem ser obtidas no repositório real do grupo e inseridas nesta seção:

1. **Repositório criado no GitHub:** print da página inicial contendo o nome do projeto e os arquivos.
2. **Branches:** print mostrando `main`, `develop` e branches de funcionalidades.
3. **Pull request:** print de um pull request com descrição, aprovação e merge.
4. **Conflito:** print da tela de conflito ou dos marcadores exibidos pelo Git.
5. **Resolução:** print do commit `2be62f4 — fix/titulo-duplicado` e do pull request número 4.
6. **Histórico:** print do gráfico de commits ou do comando `git log --oneline --decorate --graph --all`.
7. **Tag/release:** print da tag `v1.0.0` ou da release publicada.

Os prints devem mostrar, sempre que possível, o nome do repositório, a branch e o resultado da operação. Recomenda-se ocultar tokens, senhas, e-mails pessoais ou outros dados que não sejam necessários para a avaliação.

---

11. Conclusão

A atividade demonstrou, de forma prática, como o Git e o GitHub podem apoiar a Gestão de Configuração de Software. O versionamento permitiu acompanhar a evolução do projeto e recuperar estados anteriores. As branches possibilitaram que os integrantes trabalhassem de maneira paralela, reduzindo a interferência entre as atividades. Os pull requests criaram um ponto formal de revisão antes da integração e contribuíram para melhorar a qualidade do código.

A simulação do conflito foi especialmente importante porque mostrou que alterações paralelas podem atingir o mesmo artefato e que a integração exige análise do contexto. A remoção do título duplicado em `src/index.html` foi registrada no commit `2be62f4`, preservando a rastreabilidade da decisão.

As convenções de commit, o versionamento semântico e o uso de tags também facilitaram a identificação das versões entregues. Por fim, a estratégia de backup e recuperação mostrou que o repositório remoto, as tags e os bundles podem reduzir os riscos de perda de trabalho.

Conclui-se que a GCS não é apenas o armazenamento de arquivos em um repositório. Ela é um conjunto de práticas que organiza mudanças, responsabilidades, versões e critérios de integração, tornando o desenvolvimento colaborativo mais previsível, auditável e seguro.

Referências

- CHACON, Scott; STRAUB, Ben. *Pro Git*. Apress.
- GITHUB. *GitHub Documentation: About Git and GitHub*. Disponível em: https://docs.github.com/. Acesso em: novembro de 2026.
- SEMVER. *Semantic Versioning 2.0.0*. Disponível em: https://semver.org/. Acesso em: novembro de 2026.
- CONVENTIONAL COMMITS. *Conventional Commits Specification*. Disponível em: https://www.conventionalcommits.org/. Acesso em: novembro de 2026.